Sistema de Arquivos com FUSE: Desenvolvimento Parcial de um Arquivo Virtual de Monitoramento

Mateus Borges Rosenzweig[1]

[1]Engenharia de Computação – Universidade Federal do Ceará
Quixadá – Ceará – Brasil

mateus.rosenzweig@alu.ufc.br

20 de outubro de 2026

Abstract. *This paper reports the partial development of a FUSE filesystem based on the libfuse passthrough example. A read-only virtual file, status.txt, reports directory creations, file openings, and read requests. Its contents are generated in memory for each opening. Direct callback tests checked partial reads, concurrent counters, directory listing, and write protection. The final stage will add byte counters and a virtual history of recent operations, followed by integration tests through a real FUSE mount. These extensions have not yet been implemented.*

Resumo. *Este trabalho apresenta o desenvolvimento parcial de um sistema de arquivos com FUSE baseado no exemplo passthrough da libfuse. O arquivo virtual status.txt informa criações de diretórios, aberturas e pedidos de leitura, com conteúdo gerado em memória a cada abertura. Testes diretos dos callbacks verificaram leitura parcial, concorrência, listagem e proteção contra escrita. Para a entrega final, são propostas a contagem de bytes e a consulta a um histórico virtual de operações, além de testes por uma montagem real. Essas extensões ainda não foram implementadas.*

1. Introdução

Sistemas de arquivos permitem que aplicações acessem dados por meio de nomes, diretórios e operações conhecidas, como abrir, ler e escrever. Essas operações fazem parte do uso cotidiano do computador, mas os programas que as utilizam geralmente não precisam conhecer a forma como os dados são armazenados. Estudar essa interface ajuda a compreender a relação entre aplicações e sistema operacional.

A atividade foi proposta pela disciplina de Projeto e Implementação de Sistemas Operacionais, com o objetivo de entender e implementar um sistema de arquivos com funcionalidades próprias. Para isso, foi utilizada a interface FUSE, que permite atender a operações de arquivos em um programa executado em espaço de usuário. A biblioteca libfuse auxilia na comunicação desse programa com o componente FUSE do kernel [Projeto libfuse sdb].

O objetivo geral do projeto é adaptar o exemplo `passthrough.c` para disponibilizar informações sobre seu funcionamento por meio de arquivos virtuais. Esta entrega parcial apresenta a primeira etapa, cuja funcionalidade implementada é o arquivo `status.txt`, acessível na raiz do ponto de montagem. Ele apresenta três contadores:

diretórios criados com sucesso, aberturas de arquivos reais bem-sucedidas e pedidos de leitura recebidos pelo callback.

A funcionalidade pode ser observada com comandos comuns, como `cat`. Nesta etapa, o foco é compreender os callbacks e sua integração. A entrega final deverá ampliar o monitoramento com volume de dados transferidos e histórico de operações, conforme o plano da Seção 6.

2. Fundamentação Teórica

2.1. Arquivos, diretórios e operações

Um arquivo possui conteúdo e metadados, como tipo, tamanho e permissões. Diretórios organizam entradas por nomes. Consultar atributos, listar entradas e ler conteúdo são operações distintas nessa interface.

Quando um arquivo real é aberto pelo processo, a função `open` retorna um descritor. Esse número identifica a abertura e pode ser usado em operações posteriores. No passthrough, o descritor é guardado no campo `fh` de `fuse_file_info`. A implementação pode, assim, reutilizá-lo ao atender leituras e fechar o recurso ao final [Projeto libfuse sda].

2.2. FUSE e a biblioteca libfuse

FUSE significa *Filesystem in Userspace*. Seu funcionamento envolve um componente no kernel e um programa em espaço de usuário. A libfuse oferece uma API de alto nível, baseada em caminhos, e uma API de baixo nível, baseada em inodes. Este projeto utiliza a primeira [Projeto libfuse sdb].

O ponto de montagem é o diretório pelo qual o sistema de arquivos fica acessível. Quando uma aplicação acessa um caminho dentro dele, o kernel pode encaminhar pedidos ao programa FUSE. A libfuse chama as funções registradas em `fuse_operations`, chamadas de callbacks. O programa define como cada pedido será atendido e devolve o resultado.

Um callback pode ter um nome escolhido pelo programador, mas deve respeitar a assinatura da operação. Por exemplo, o campo `.mkdir` pode apontar para `xmp_mkdir` ou para outra função compatível. Isso permite alterar o comportamento de criação de diretórios sem inventar uma nova operação no protocolo. Na API de alto nível, erros são normalmente devolvidos como valores negativos, como `-ENOENT` para um caminho inexistente [Projeto libfuse sdc].

2.3. O exemplo passthrough

O exemplo `passthrough.c` repassa solicitações para funções que operam no sistema de arquivos existente. Entre elas estão `lstat`, `mkdir`, `pread` e `pwrite`. Na forma utilizada, ele espelha a hierarquia iniciada em `/` [Projeto libfuse sda]. Assim, se estiver montado em `/tmp/meufs`, o caminho `/tmp/meufs/etc/hostname` corresponde ao arquivo real `/etc/hostname`.

Essa base permite concentrar o estudo nos callbacks. Alterações nos caminhos espelhados podem modificar arquivos reais, conforme as permissões do processo. O monitoramento foi adicionado como exceção ao encaminhamento.

3. Implementação da Etapa Parcial

3.1. Organização do arquivo virtual

O caminho reservado é `/status.txt`, considerando a raiz do sistema montado. A função auxiliar `is_status` compara o caminho recebido com esse nome. Quando há correspondência, o callback atende ao arquivo virtual. Nos demais casos, continua o fluxo do passthrough.

Esse arquivo não é criado no disco. Seu texto é produzido pela função `make_status`, que usa `snprintf` para reunir os contadores. A estrutura `status_snapshot` guarda o texto em um vetor de 256 bytes e registra seu comprimento. Esse espaço atende às três mensagens e aos valores dos contadores utilizados.

A Tabela 1 resume as principais adaptações. Apenas implementar a leitura seria insuficiente: o caminho também precisa ser reconhecido na consulta de atributos e aparecer na listagem.

Tabela 1. Participação dos callbacks no arquivo virtual.

Callback	Adaptação realizada
`getattr`	Informa arquivo regular, tamanho e permissões 0444.
`readdir`	Inclui o nome na raiz da montagem.
`open`	Aceita leitura e guarda uma cópia do texto.
`read`	Copia o intervalo solicitado para o buffer.
`release`	Libera a memória associada à abertura.
`write`	Recusa alterações no conteúdo virtual.

Na listagem da raiz, o nome é inserido por meio de `filler`. Se houver um arquivo real com o mesmo nome nessa raiz, sua entrada é omitida para evitar duplicação. Essa escolha reserva o nome para o monitoramento. Um arquivo chamado `status.txt` dentro de outro diretório continua seguindo o comportamento normal.

3.2. Contagem das operações

Os contadores foram declarados com o tipo `atomic_uint_fast64_t`. O uso de operações atômicas evita a perda de incrementos quando mais de uma thread atende solicitações. O contador de diretórios é incrementado somente depois de `mkdir` ter sucesso. O trecho abaixo foi extraído dessa parte do callback:

```
res = mkdir(path, mode);

if (res == -1)
    return -errno;

atomic_fetch_add(&directories_created, 1);
return 0;
```

Aberturas reais são contadas após o sucesso nos callbacks `open` e `create`. Já o contador de leitura é incrementado na entrada do fluxo de arquivos reais em `xmp_read`, incluindo pedidos que terminem em erro. Essa diferença precisa ser considerada na interpretação dos números: dois contadores representam sucessos e o terceiro representa solicitações ao callback.

O próprio arquivo virtual é tratado antes dos incrementos de abertura e leitura. Consultar `status.txt`, portanto, não aumenta esses contadores. Os dados permanecem apenas em memória e são reiniciados quando o processo do sistema de arquivos é iniciado novamente.

3.3. Abertura, leitura e liberação

A cada abertura do arquivo virtual, o programa aloca uma `status_snapshot` e preenche seu texto. O endereço da estrutura é armazenado em `fi->fh`. Nesse caso, esse campo guarda um ponteiro convertido para o tipo inteiro aceito pela API, enquanto para os arquivos reais continua guardando um descritor.

A decisão de manter uma cópia por abertura evita misturar trechos de textos produzidos em momentos diferentes. Se outra thread atualizar os contadores durante uma leitura, a abertura atual continuará utilizando sua cópia. Para observar novos valores, o leitor deve fechar e abrir o arquivo novamente.

A leitura considera `size`, que é a quantidade solicitada, e `offset`, que indica a posição inicial. Depois de obter a cópia correspondente à abertura, o callback executa:

```
if (offset < 0)
    return -EINVAL;
if ((uintmax_t)offset >= snapshot->length)
    return 0;
if (size > snapshot->length - (size_t)offset)
    size = snapshot->length - (size_t)offset;
memcpy(buf, snapshot->text + (size_t)offset, size);
return (int)size;
```

O retorno zero informa o fim do arquivo. Limitar o tamanho da cópia evita acessar posições além do texto. Cada pedido recebe apenas o intervalo disponível a partir da posição solicitada.

A abertura virtual também configura `direct_io = 1` e `keep_cache = 0`. Isso permite que novas aberturas consultem o texto atualizado, sem reutilizar o cache de dados do arquivo. Ao encerrar a abertura, `release` libera a memória com `free`. Usar `close` nesse caso seria incorreto, pois o recurso guardado é uma estrutura em memória.

3.4. Proteção e tratamento de erros

Os atributos apresentam o arquivo como somente leitura, mas a implementação também recusa operações que poderiam alterá-lo. Escrita, truncamento, remoção e renomeação envolvendo o caminho reservado são bloqueados nos callbacks. A criação de outra entrada nesse mesmo caminho retorna um erro de existência.

Também foram tratados os callbacks opcionais presentes na base, incluindo atributos estendidos, consulta com `statx` e cópia por intervalo. Quando o arquivo virtual é a origem de `copy_file_range`, a função retorna operação não suportada, permitindo que um cliente compatível tente copiar por leitura e escrita. O objetivo é evitar que essas rotas usem o recurso virtual como se ele fosse um descritor de arquivo real.

4. Metodologia da Avaliação Parcial

A implementação foi compilada com GCC 16.2.1 e libfuse 3.18.3. O código de referência pertence ao repositório libfuse, na revisão `fe66006f`, com as modificações locais

descritas neste artigo. Foram verificadas compilações com e sem as opções de callbacks condicionais, incluindo `HAVE_STATX` e `HAVE_COPY_FILE_RANGE`.

A avaliação foi feita por um programa de testes em C que inclui o arquivo da implementação e chama seus callbacks diretamente. As operações reais utilizaram um diretório temporário. Para verificar a listagem da raiz e a colisão com um nome existente, o teste redirecionou a chamada de abertura do diretório raiz para uma pasta de teste. Nenhum arquivo real da raiz precisou ser substituído.

Foram usados AddressSanitizer e UndefinedBehaviorSanitizer para detectar problemas de memória e comportamento indefinido durante os testes. A detecção de vazamentos foi desativada porque LeakSanitizer não funcionou sob o mecanismo de execução disponível. Essa configuração não permite concluir que o programa esteja livre de vazamentos.

O ambiente de execução dos testes não disponibilizou `/dev/fuse`. Dessa forma, a avaliação não incluiu uma montagem real nem a comunicação completa entre aplicação e kernel. Para essa etapa posterior, o procedimento proposto é compilar o programa, montá-lo e acessar o arquivo, por exemplo:

```
mkdir -p /tmp/meufs
./passthrough-status -f /tmp/meufs
# Em outro terminal:
cat /tmp/meufs/status.txt
cat /tmp/meufs/etc/hostname
cat /tmp/meufs/status.txt
```

Esses comandos são um roteiro de demonstração, não um registro de execução usado para produzir os resultados da próxima seção. A desmontagem pode ser feita com `fusermount3 -u /tmp/meufs`.

5. Resultados Parciais e Discussão

A compilação foi concluída nas duas configurações verificadas. Nos testes diretos, o arquivo virtual apresentou atributos de arquivo regular com permissões 0444, aceitou leitura e recusou acesso de escrita. A Tabela 2 apresenta os principais resultados.

Tabela 2. Resultados observados nos testes diretos dos callbacks.

Verificação	Resultado observado
Leitura de um byte por vez	Texto reconstruído e retorno zero no final.
Atualização após abrir	Abertura antiga preservada; nova abertura atualizada.
Criação repetida	Um incremento no primeiro sucesso; erro na repetição.
Listagem da raiz	Uma entrada virtual, inclusive com nome real coincidente.
Operações de alteração	Pedidos de escrita e alteração do caminho recusados.
Concorrência	800 aberturas e leituras adicionais contabilizadas.
Arquivos reais	Criação, escrita, leitura e remoção funcionaram no teste.

O teste concorrente utilizou oito threads. Cada uma realizou 100 ciclos de aber-

tura, leitura de um byte e liberação de um arquivo real. Antes desses ciclos, havia uma abertura bem-sucedida por `create` e dois pedidos de leitura, um deles sobre um caminho inexistente. Ao final, foram verificados 801 registros de abertura e 802 pedidos de leitura. Esses totais correspondem às ações executadas pelo teste, e não a uma medição de desempenho.

A leitura do arquivo virtual foi repetida na mesma abertura depois de alterar os contadores. O texto permaneceu igual ao obtido no início. Após fechar e reabrir, os novos valores apareceram. Também foram verificados tamanho zero de leitura, posição negativa e posição além do fim do conteúdo. Isso confirma o tratamento desses casos no callback testado.

O comportamento observado atende ao objetivo de expor informações por meio de um arquivo gerado em memória. Entretanto, o contador de leitura não representa todas as leituras feitas pelas aplicações: o cache pode atender a um pedido sem chamar o programa FUSE. Também não existe uma relação obrigatória entre um comando `cat` e um único incremento.

Há outras limitações na versão atual. Os contadores não são persistidos, o nome na raiz é fixo e o passthrough continua espelhando o sistema existente. Cada contador é lido atomicamente, mas a leitura dos três não constitui uma única transação; sob concorrência, os valores podem corresponder a instantes próximos. Depois de gerado, o texto de cada abertura permanece estável.

Os testes não avaliaram tempo de resposta, consumo de memória ou comportamento sob carga prolongada. Também não verificaram todos os detalhes herdados do exemplo original. Os resultados sustentam as funções exercitadas diretamente, enquanto a montagem real permanece necessária para completar a avaliação.

6. Plano para a Entrega Final

A próxima etapa deverá transformar o relatório de contadores em um monitor de operações. As funcionalidades descritas nesta seção são propostas para a entrega final e ainda não fazem parte do código avaliado.

6.1. Contagem de bytes e histórico virtual

O arquivo `status.txt` deverá informar também o total de bytes lidos e escritos pelos callbacks. O incremento usará o retorno positivo de `pread` ou `pwrite`, em vez do tamanho solicitado. Assim, leituras parciais terão seu volume efetivo contabilizado, enquanto falhas e leituras no fim do arquivo não acrescentarão bytes. A contagem continuará limitada às operações que chegam ao programa FUSE.

Será acrescentado o arquivo virtual `/historico.txt`, somente para leitura, com os últimos 100 registros de criação de diretório, abertura, leitura e escrita. Cada registro deverá informar um número sequencial, a operação, o caminho, o resultado e a quantidade de bytes, quando aplicável. O conteúdo dos arquivos acessados não será registrado. A consulta poderá ser feita com `cat /tmp/meufs/historico.txt`.

Os registros serão mantidos em um vetor circular: quando atingir o limite, o registro mais antigo será substituído. Um mutex deverá proteger a atualização e a cópia do histórico entre threads. Cada abertura do arquivo virtual receberá uma cópia estável,

seguindo a estratégia já usada no status. Consultas aos dois arquivos virtuais serão excluídas do monitoramento. O histórico ficará em memória e será reiniciado junto com o processo.

6.2. Verificação da versão final

A validação deverá incluir montagem real e comandos como `mkdir`, `ls`, `stat` e `cat`. Os testes diretos verificarão a soma dos bytes retornados e a retenção dos 100 registros mais recentes após 101 operações monitoradas. Também serão repetidos os testes de leitura em partes, proteção contra escrita e acesso concorrente. A entrega final deverá apresentar evidências dessa integração e comparar os resultados com os requisitos propostos.

7. Considerações Parciais

A primeira etapa disponibilizou o arquivo virtual de monitoramento e integrou consulta de atributos, listagem, abertura, leitura e liberação. Os testes diretos confirmaram os comportamentos avaliados, incluindo as atualizações concorrentes dos contadores. A validação pela montagem real permanece pendente.

Essa base será reutilizada na entrega final para acrescentar a contagem de bytes e o histórico virtual. A ampliação permitirá observar tanto os totais acumulados quanto as operações recentes, mantendo o acesso por comandos comuns do sistema. Os resultados apresentados nesta parcial se referem apenas à implementação atual de `status.txt`.

Referências

Projeto libfuse (s.d.a). Código-fonte do exemplo passthrough.c. `https://github.com/libfuse/libfuse/blob/master/example/passthrough.c`. Acesso em: 29 out. 2026. Substituir master pelo commit utilizado no projeto.

Projeto libfuse (s.d.b). libfuse: Readme e documentação do projeto. `https://github.com/libfuse/libfuse`. Acesso em: 29 out. 2026.

Projeto libfuse (s.d.c). Referência da estrutura fuse_operations. `https://libfuse.github.io/doxygen/structfuse__operations.html`. Acesso em: 29 out. 2026.